

Machine Learning (CAI-262)

Objectives:

-
- Reinforcement Learning
 - Algorithm basics
 - Types of RL
-

Reinforcement Learning:

Reinforcement learning (RL) is a machine learning (ML) technique that trains software to make decisions to achieve the most optimal results. It mimics the trial-and-error learning process that humans use to achieve their goals. Software actions that work towards your goal are reinforced, while actions that detract from the goal are ignored.

RL algorithms use a reward-and-punishment paradigm as they process data. They learn from the feedback of each action and self-discover the best processing paths to achieve final outcomes. The algorithms are also capable of delayed gratification. The best overall strategy may require short-term sacrifices, so the best approach they discover may include some punishments or backtracking along the way. RL is a powerful method to help artificial intelligence (AI) systems achieve optimal outcomes in unseen environments.

Benefits of reinforcement learning

There are many benefits to using reinforcement learning (RL). However, these three often stand out.

Excels in complex environments

RL algorithms can be used in complex environments with many rules and dependencies. In the same environment, a human may not be capable of determining the best path to take, even with superior knowledge of the environment. Instead, model-free RL algorithms adapt quickly to continuously changing environments and find new strategies to optimize results.

Requires less human interaction

In traditional ML algorithms, humans must label data pairs to direct the algorithm. When you use an RL algorithm, this isn't necessary. It learns by itself. At the same time, it offers mechanisms to integrate human feedback, allowing for systems that adapt to human preferences, expertise, and corrections.

Optimizes for long-term goals

RL inherently focuses on long-term reward maximization, which makes it apt for scenarios where actions have prolonged consequences. It is particularly well-suited for real-world situations where feedback isn't immediately available for every step, since it can learn from delayed rewards. RL can be used to optimize long-term energy efficiency and cost. With appropriate architectures, RL agents can also generalize their learned strategies across similar but not identical tasks.

How does reinforcement Learning work?

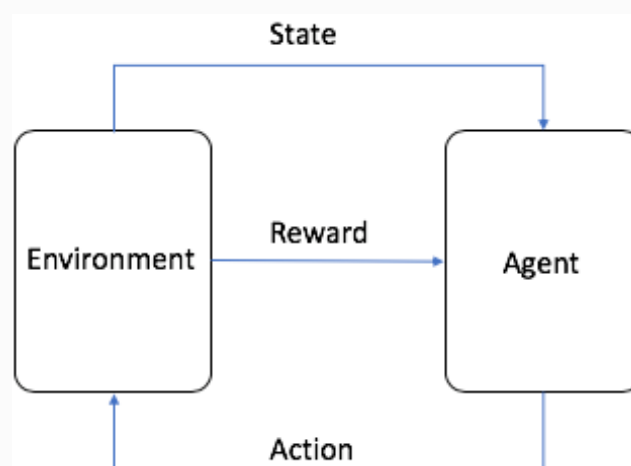
The learning process of reinforcement learning (RL) algorithms is similar to animal and human reinforcement learning in the field of behavioural psychology. For instance, a child may discover that they receive parental praise when they help a sibling or clean but receive negative reactions when they throw toys or yell. Soon, the child learns which combination of activities results in the end reward.

An RL algorithm mimics a similar learning process. It tries different activities to learn the associated negative and positive values to achieve the end reward outcome. In reinforcement learning, there are a few key concepts to familiarize yourself with:

- The **agent** is the ML algorithm (or the autonomous system)
- The **environment** is the adaptive problem space with attributes such as variables, boundary values, rules, and valid actions
- The **action** is a step that the RL agent takes to navigate the environment
- The **state** is the environment at a given point in time
- The **reward** is the positive, negative, or zero value—in other words, the reward or punishment—for taking an action
- The **cumulative reward** is the sum of all rewards or the end value
- The **policy** is the strategy an agent adopts to reach its goal optimally.

Algorithm basics

Reinforcement learning is based on the **Markov decision process**, a mathematical modelling of decision-making that uses discrete time steps. At every step, the agent takes a new action that results in a new environment state. Similarly, the current state is attributed to the sequence of previous actions. Through trial and error in moving through the environment, the agent builds a set of if-then rules or policies. The policies help it decide which action to take next for optimal cumulative reward. The agent must also choose between further environment exploration to learn new state-action rewards or select known high-reward actions from a given state. This is called the **exploration-exploitation** trade-off.



Types of reinforcement learning algorithms

There are various algorithms used in reinforcement learning (RL)—such as Q-learning, policy gradient methods, Monte Carlo methods, and temporal difference learning. Deep RL is the application of deep neural networks to reinforcement learning. One example of a deep RL algorithm is Trust Region Policy Optimization (TRPO). All these algorithms can be grouped into two broad categories.

Model-based RL

Model-based RL is typically used when environments are well-defined and unchanging and where real-world environment testing is difficult. The agent first builds an internal representation (model) of the environment. It uses this process to build this model:

1. It takes actions within the environment and notes the new state and reward value
2. It associates the action-state transition with the reward value.

Once the model is complete, the agent simulates action sequences based on the probability of optimal cumulative rewards. It then further assigns values to the action sequences themselves. The agent thus develops different strategies within the environment to achieve the desired end goal.

Example

Consider a robot learning to navigate a new building to reach a specific room. Initially, the robot explores freely and builds an internal model (or map) of the building. For instance, it might learn that it encounters an elevator after moving forward 10 meters from the main entrance. Once it builds the map, it can build a series of shortest-path sequences between different locations it visits frequently in the building.

Model-free RL

Model-free RL is best to use when the environment is large, complex, and not easily describable. It's also ideal when the environment is unknown and changing, and environment-based testing does not come with significant downsides. The agent doesn't build an internal model of the environment and its dynamics. Instead, it uses a trial-and-error approach within the environment. It scores and notes state-action pairs—and sequences of state-action pairs—to develop a policy.

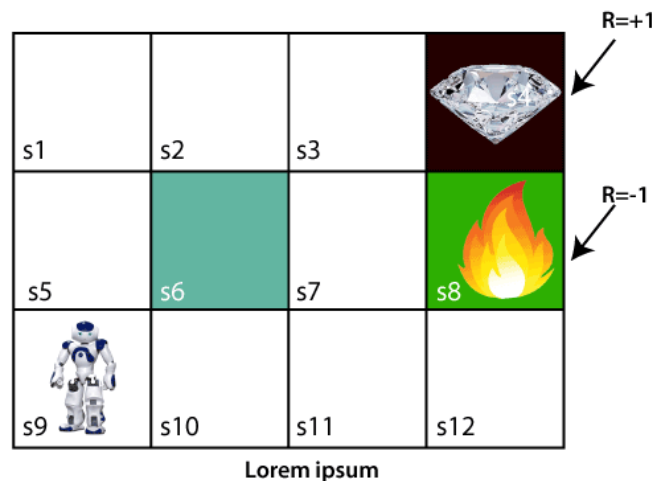
Example

Consider a self-driving car that needs to navigate city traffic. Roads, traffic patterns, pedestrian behaviour, and countless other factors can make the environment highly dynamic and complex. AI teams train the vehicle in a simulated environment in the initial stages. The vehicle takes actions based on its current state and receives rewards or penalties. Over time, by driving millions of miles in different virtual scenarios, the vehicle learns which actions are best for each state without explicitly modelling the entire traffic dynamics. When introduced in the real world, the vehicle uses the learned policy but continues to refine it with new data.

To understand the working process of the RL, we need to consider two main things:

- **Environment:** It can be anything such as a room, maze, football ground, etc.
- **Agent:** An intelligent agent such as AI robot.

Let's take an example of a maze environment that the agent needs to explore. Consider the below image:

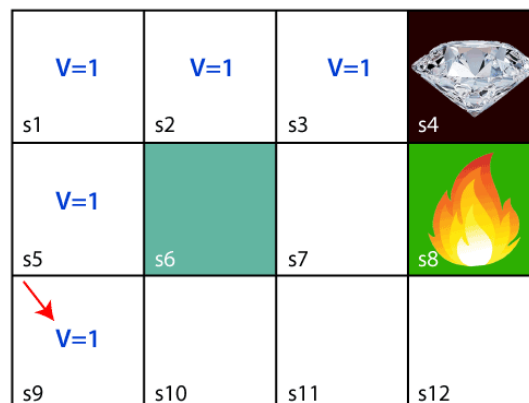


In the above image, the agent is at the very first block of the maze. The maze is consisting of an S_6 block, which is a **wall**, S_8 a **fire pit**, and S_4 a **diamond block**.

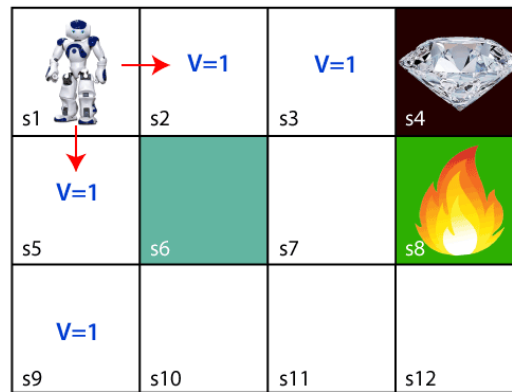
The agent cannot cross the S_6 block, as it is a solid wall. If the agent reaches the S_4 block, then get the **+1 reward**; if it reaches the fire pit, then gets **-1 reward point**. It can take four actions: **move up, move down, move left, and move right**.

The agent can take any path to reach to the final point, but he needs to make it in possible fewer steps. Suppose the agent considers the path **S9-S5-S1-S2-S3**, so he will get the +1-reward point.

The agent will try to remember the preceding steps that it has taken to reach the final step. To memorize the steps, it assigns 1 value to each previous step. Consider the below step:



Now, the agent has successfully stored the previous steps assigning the 1 value to each previous block. But what will the agent do if he starts moving from the block, which has 1 value block on both sides? Consider the below diagram:



It will be a difficult condition for the agent whether he should go up or down as each block has the same value. So, the above approach is not suitable for the agent to reach the destination. Hence to solve the problem, we will use the **Bellman equation**, which is the main concept behind reinforcement learning.

The Bellman Equation

The Bellman equation was introduced by the Mathematician **Richard Ernest Bellman in the year 1953**, and hence it is called as a Bellman equation. It is associated with dynamic programming and used to calculate the values of a decision problem at a certain point by including the values of previous states. It is a way of calculating the value functions in dynamic programming or environment that leads to modern reinforcement learning.

The key-elements used in Bellman equations are:

- Action performed by the agent is referred to as "a"
- State occurred by performing the action is "s."
- The reward/feedback obtained for each good and bad action is "R."
- A discount factor is Gamma "γ."

The Bellman equation can be written as:

$$1. \quad V(s) = \max [R(s,a) + \gamma V(s')]$$

Where,

V(s)= value calculated at a particular point.

R(s,a) = Reward at a particular state s by performing an action.

γ = Discount factor

V(s') = The value at the previous state.

In the above equation, we are taking the max of the complete values because the agent tries to find the optimal solution always.

So now, using the Bellman equation, we will find value at each state of the given environment. We will start from the block, which is next to the target block.

For 1st block:

$V(s_3) = \max [R(s,a) + \gamma V(s')]$, here $V(s') = 0$ because there is no further state to move.

$V(s_3) = \max [R(s,a)] \Rightarrow V(s_3) = \max [1] \Rightarrow V(s_3) = 1.$

For 2nd block:

$V(s_2) = \max [R(s,a) + \gamma V(s')]$, here $\gamma = 0.9$ (lets), $V(s') = 1$, and $R(s, a) = 0$, because there is no reward at this state.

$V(s_2) = \max [0.9(1)] \Rightarrow V(s_2) = \max [0.9] \Rightarrow V(s_2) = 0.9$

For 3rd block:

$V(s_1) = \max [R(s,a) + \gamma V(s')]$, here $\gamma = 0.9$ (lets), $V(s') = 0.9$, and $R(s, a) = 0$, because there is no reward at this state also.

$V(s_1) = \max [0.9(0.9)] \Rightarrow V(s_1) = \max [0.81] \Rightarrow V(s_1) = 0.81$

For 4th block:

$V(s_5) = \max [R(s,a) + \gamma V(s')]$, here $\gamma = 0.9$ (lets), $V(s') = 0.81$, and $R(s, a) = 0$, because there is no reward at this state also.

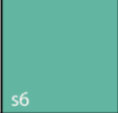
$$V(s_5) = \max[0.9(0.81)] \Rightarrow V(s_5) = \max[0.73] \Rightarrow V(s_5) = 0.73$$

For 5th block:

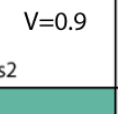
$V(s_9) = \max [R(s,a) + \gamma V(s')]$, here $\gamma = 0.9$ (lets), $V(s') = 0.73$, and $R(s, a) = 0$, because there is no reward at this state also.

$$V(s_9) = \max[0.9(0.73)] \Rightarrow V(s_9) = \max[0.66] \Rightarrow V(s_9) = 0.66$$

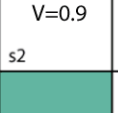
Consider the below image:

V=0.81 s1	V=0.9 s2	V=1 s3	 s4
V=0.73 s5		s7	 s8
 V=0.66 s9	s10	s11	s12

Now, we will move further to the 6th block, and here agent may change the route because it always tries to find the optimal path. So now, let's consider from the block next to the fire pit.

V=0.81 s1	V=0.9 s2	V=1 s3	 s4
V=0.73 s5		 s7	 s8
V=0.66 s9	s10	 s11	s12

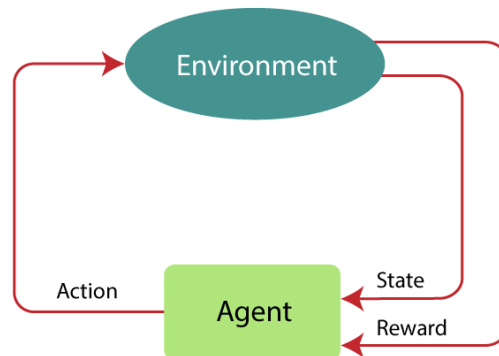
Now, the agent has three options to move; if he moves to the blue box, then he will feel a bump if he moves to the fire pit, then he will get the -1 reward. But here we are taking only positive rewards, so for this, he will move to upwards only. The complete block values will be calculated using this formula. Consider the below image:

V=0.81 s1	V=0.9 s2	V=1 s3	 s4
V=0.73 s5		V=0.9 s7	 s8
V=0.66 s9	V=0.73 s10	V=0.81 s11	V=0.73 s12

Markov Decision Process

Markov Decision Process or MDP, is used to **formalize the reinforcement learning problems**. If the environment is completely observable, then its dynamic can be modeled as a **Markov Process**. In MDP,

the agent constantly interacts with the environment and performs actions; at each action, the environment responds and generates a new state.



MDP is used to describe the environment for the RL, and almost all the RL problem can be formalized using MDP.

MDP contains a tuple of four elements (S, A, P_a , R_a):

- A set of finite States S
- A set of finite Actions A
- Rewards received after transitioning from state S to state S', due to action a.
- Probability P_a .

MDP uses **Markov property**, and to better understand the MDP, we need to learn about it.

Markov Property:

It says that *"If the agent is present in the current state S_1 , performs an action a_1 and move to the state s_2 , then the state transition from s_1 to s_2 only depends on the current state and future action and states do not depend on past actions, rewards, or states."*

Or, in other words, as per Markov Property, the current state transition does not depend on any past action or state. Hence, MDP is an RL problem that satisfies the Markov property. Such as in a **Chess game, the players only focus on the current state and do not need to remember past actions or states.**

Finite MDP:

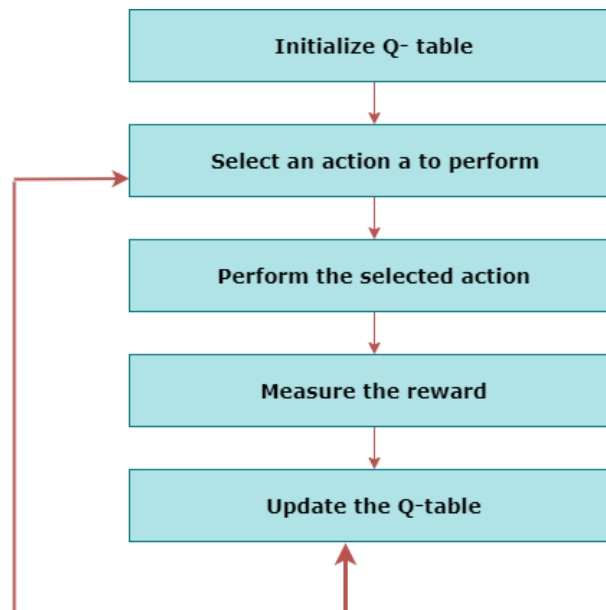
A finite MDP is when there are finite states, finite rewards, and finite actions. In RL, we consider only the finite MDP.

Markov Process:

Markov Process is a memoryless process with a sequence of random states $S_1, S_2, \dots, S_t$ that uses the Markov Property. Markov process is also known as Markov chain, which is a tuple (S, P) on state S and transition function P. These two components (S and P) can define the dynamics of the system.

Q-Learning:

- The Q stands for **quality** in **Q-learning**, which means it specifies the quality of an action taken by the agent.
- It learns the value function $Q(S, a)$, which means how good to take action "a" at a particular state "s."
- The below flowchart explains the working of Q- learning:



- Q-learning is a popular model-free reinforcement learning algorithm based on the Bellman equation.
- The main objective of Q-learning is to learn the policy which can inform the agent that what actions should be taken for maximizing the reward under what circumstances.
- It is an **off-policy RL** that attempts to find the best action to take at a current state.
- The goal of the agent in Q-learning is to maximize the value of Q.
- The value of Q-learning can be derived from the Bellman equation. Consider the Bellman equation given below:

$$\text{New } Q(s, a) = Q(s, a) + \alpha [R(s, a) + \gamma \max_{a'} Q'(s', a') - Q(s, a)]$$

- New Q Value for that state and the action
- Learning Rate
- Reward for taking that action at that state
- Current Q Values
- Maximum expected future reward given the new state (s') and all possible actions at that new state.
- Discount Rate

The above formula is used to estimate the Q-values in Q-Learning.

Q-table:

A Q-table or matrix is created while performing the Q-learning. The table follows the state and action pair, i.e., [s, a], and initializes the values to zero. After each action, the table is updated, and the q-values are stored within the table.

The RL agent uses this Q-table as a reference table to select the best action based on the q-values.

Example

Step 1: initialize the Q-Table

We will first build a Q-table. There are n columns, where n = number of actions. There are m rows, where m = number of states. We will initialise the values at 0.

Actions : ↑ → ↓ ←

Start	0	0	0	0
Nothing / Blank	0	0	0	0
Power	0	0	0	0
Mines	0	0	0	0
END	0	0	0	0

					
					
					
					
				End	

Steps 2 and 3: choose and perform an action

This combination of steps is done for an undefined amount of time. This means that this step runs until the time we stop the training, or the training loop stops as defined in the code.

We will choose an action (a) in the state (s) based on the Q-Table. But, as mentioned earlier, when the episode initially starts, every Q-value is 0.

We'll use something called the **epsilon greedy strategy**.

In the beginning, the epsilon rates will be higher. The robot will explore the environment and randomly choose actions. The logic behind this is that the robot does not know anything about the environment. As the robot explores the environment, the epsilon rate decreases and the robot starts to exploit the environment.

During the process of exploration, the robot progressively becomes more confident in estimating the Q-values.

For the robot example, there are four actions to choose from: up, down, left, and right. We are starting the training now — our robot knows nothing about the environment. So the robot chooses a random action, say right.

Actions : ↑ → ↓ ←

					
					
					
					
				End	

Start	0	0	0	0
Nothing / Blank	0	0	0	0
Power	0	0	0	0
Mines	0	0	0	0
END	0	0	0	0

Steps 4 and 5: evaluate

Now we have taken an action and observed an outcome and reward. We need to update the function $Q(s,a)$.

In the case of the robot game, to reiterate the scoring/reward structure is:

- **power** = +1
- **mine** = -100
- **end** = +100

New $Q(\text{start}, \text{right}) = Q(\text{start}, \text{right}) + \alpha[\text{some ... Delta value}]$

Some ... Delta value = $R(\text{start}, \text{right}) + \max(Q'(\text{nothing}, \text{down}), Q'(\text{nothing}, \text{left}), Q'(\text{nothing}, \text{right})) - Q(\text{start}, \text{right})$

Some ... Delta value = $0 + 0.9 * 0 - 0 = 0$

New $Q(\text{start}, \text{right}) = 0 + 0.1 * 0 = 0$

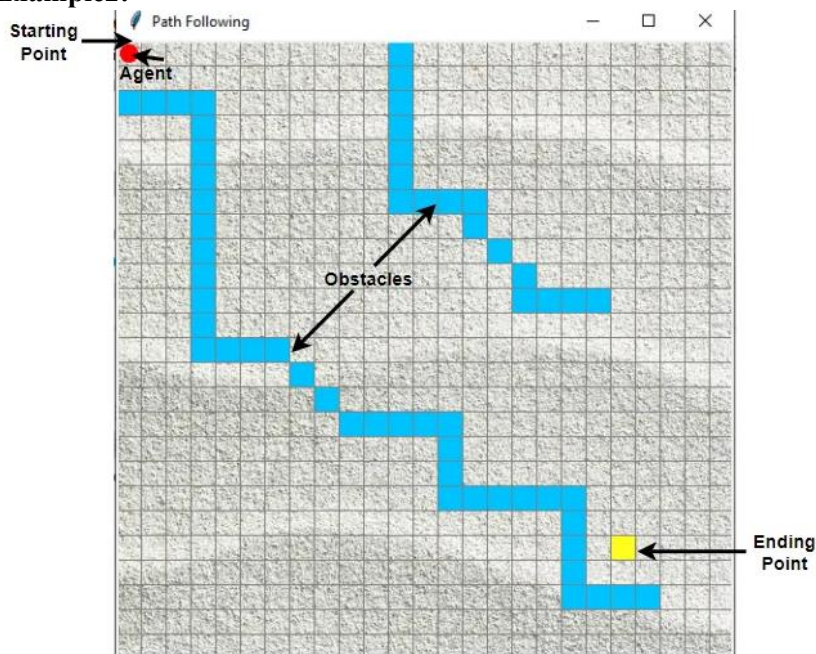
				
				
				
				
			End	

Actions : ↑ → ↓ ←

Start	0	0	0	0
Nothing / Blank	0	0	0	0
Power	0	0	0	0
Mines	0	0	0	0
END	0	0	0	0

We will repeat this again and again until the learning is stopped. In this way the Q-Table will be updated.

Example2:



In the example, I have entered the rewarding scheme as follows.

Reward when reach step closer to goal= +1

Reward when hit obstacle =-1

Reward when idle=0

Step 1: Initialize the Q-Table

First the Q-table has to be built. There are n columns, where n= number of actions. There are m rows, where m= number of states.

In our example n =Go Left, Go Right, Go Up and Go Down and m = Start, Idle, Correct Path, Wrong Path and End. First, let's initialize the values at 0.

Action	↑	↓	→	←
Start	0	0	0	0
Idle	0	0	0	0
Correct Path	0	0	0	0
Wrong Path	0	0	0	0
End	0	0	0	0

Step 2 ,3: Choose an Action and Perform an Action

The combination of steps 2 and 3 is performed for an undefined amount of time. These steps runs until the time training is stopped, or when the training loop stopped as defined in the code.

First, an action (a) in the state (s) is chosen based on the Q-Table. Note that, as mentioned earlier, when the episode initially starts, every Q-value should be 0.

Then, update the Q-values for being at the start and moving right using the Bellman equation which is stated above.

We can now update the Q-values for being at the start and moving right using the Bellman equation.

Action	↑	↓	→	←
Start	0	0	1	0
Idle	0	0	0	0
Correct Path	0	0	0	0
Wrong Path	0	0	0	0
End	0	0	0	0

Steps 4 ,5 : Measure Reward and evaluate

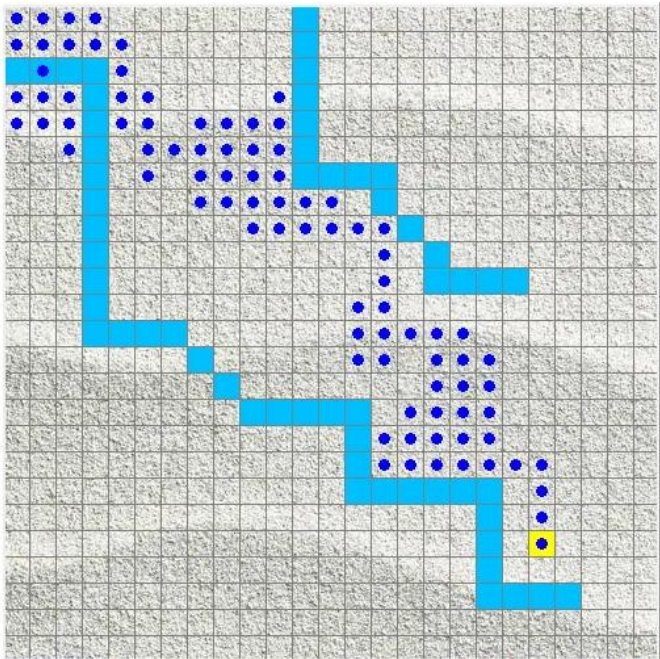
Now we have taken an action and observed an outcome and reward.

initially, we explore the agent's environment and update the Q-Table. When the Q-Table is ready, the agent start to exploit the environment and start taking better actions. Final Q-table can be like following (for example).

Action	↑	↓	→	←
Start	0	0	1	0
Idle	0	0	0	0
Correct Path	0	50	22	0
Wrong Path	15	0	18	0
End	0	0	1	0

Example final Q-table

Following are the outcomes that results the agent's shortest path towards goal after training.



Example3:

In this environment, the agent must cross the frozen lake from the start to the goal, without falling into the holes. The best strategy is to reach goals by taking the shortest path.



- The reward for reaching the goal is +1
- The reward for taking the wrong path (falling into the hole) is 0
- The reward for Idle or moving on the frozen lake is also 0.

	→	←	↑	↓
Start	0	0	0	0
Idle	0	0	0	0
Hole	0	0	0	0
End	0	0	0	0